

Temas Selectos de Física Computacional III:

Introducción a la ciencia de datos.

Clase 6: 11 Marzo 2025

Dr. Leonid Serkin

1 Introducción

En esta clase, estudiaremos el problema de dimensionalidad y el método de componentes principales aplicado al conjunto de datos de cáncer de mama.

2 Mejorando la predicción de cáncer

El conjunto de datos de cáncer de mama, accesible desde Scikit-learn, es utilizado para predecir si un tumor es maligno o benigno.

```
from sklearn.datasets import load_breast_cancer
import pandas as pd
cancer = load_breast_cancer()
df = pd.DataFrame(data=cancer.data, columns=cancer.feature_names)
df['target'] = cancer.target
target_labels = cancer.target[[10, 20, 30]]
print("labels:", target_labels)
print("clase:", cancer.target_names[target_labels])
```

En la clase pasada, estudiamos la correlación de todas las variables con el diagnóstico (`target`) y las ordenamos:

```
correlation = df.corr()
correlation_target = correlation['target']
correlation_absolute = correlation_target.abs()
correlation_sorted = correlation_absolute.sort_values(ascending=False)
print(correlation_sorted)
```

La variable más correlacionadas con `target` es:

```
worst concave points      0.793566
```

y para un valor de umbral de 0.1251 obtenemos las métricas de:

```

umbral_wc = 0.1251
df['prediction_wcp'] = (df['worst concave points'] >= umbral_wc).astype(int)
cm_wc = confusion_matrix(df['target'], df['prediction_wcp'])
print(round(accuracy_score(df['target'], df['prediction_wcp']), 4))
print(round(precision_score(df['target'], df['prediction_wcp']), 4))
print(round(recall_score(df['target'], df['prediction_wcp']), 4))
print(round(f1_score(df['target'], df['prediction_wcp']), 4))

```

lo que nos da:

```

Exactitud:  0.1002
Precisión:  0.1525
Recall:     0.0952
F1-score:   0.1172

```

Nuestro modelo "a ojo" usando una variable **no es útil en el diagnóstico**. La precisión es baja, lo que significa que muchas de las predicciones de tumores malignos son incorrectas (falsos positivos). El Recall es extremadamente bajo, lo que significa que el modelo no detecta la mayoría de los casos malignos (altos falsos negativos).

En muchos problemas de la vida real no siempre hay una variable individualmente fuerte para separar entre clases. También puede ser que las características pueden estar correlacionadas, lo que significa que contienen información redundante. Y también algunas variables pueden contener información complementaria que una sola no captura. Por lo tanto **se debe mejorar el modelo**, probando otras técnicas como usar PCA con más componentes, ajustar los umbrales o probar un modelo de clasificación más avanzado.

3 Dimensionalidad

A medida que aumenta el número de variables o dimensiones en un conjunto de datos, la cantidad de datos necesarios para obtener un resultado estadísticamente significativo aumenta exponencialmente. Esto puede llevar a problemas como sobreajuste, aumento del tiempo de cálculo y reducción de la precisión de los modelos de aprendizaje automático. Este fenómeno se conoce como la **maldición de la dimensionalidad**, que se refiere a los problemas que surgen al trabajar con datos con muchas variables.

A medida que aumenta el número de dimensiones, el número de posibles combinaciones de variables de entrada aumenta exponencialmente, lo que dificulta obtener una muestra representativa de los datos. Realizar tareas como clasificación se vuelve más costoso porque los algoritmos necesitan procesar un espacio de características mucho más grande, lo que aumenta el tiempo de cómputo y la complejidad. Además, **algunos algoritmos de aprendizaje automático pueden ser sensibles al número de dimensiones**, requiriendo más datos para lograr el mismo nivel de precisión que con datos de menor dimensionalidad.

La **reducción de dimensionalidad** es un tipo de técnica de extracción de características que tiene como objetivo reducir el número de variables de entrada mientras se conserva la mayor cantidad posible de la información original.

4 Análisis de componentes principales (PCA)

El análisis de componentes principales (PCA) fue introducido por el matemático Karl Pearson en 1901. Funciona bajo la condición de que, al mapear los datos de un espacio de mayor dimensión a un espacio de menor dimensión, **la varianza de los datos en el espacio de menor dimensión debe ser máxima**.

El PCA es un procedimiento estadístico que utiliza una transformación ortogonal para convertir un conjunto de variables correlacionadas en un conjunto de variables no correlacionadas. Es la herramienta más utilizada en el análisis exploratorio de datos y en el aprendizaje automático para modelos predictivos.

El PCA se utiliza para reducir la dimensionalidad de un conjunto de datos al encontrar un nuevo conjunto de variables, más pequeño que el conjunto original, que retenga la mayor parte de la información de la muestra y sea útil para la regresión y clasificación de datos.

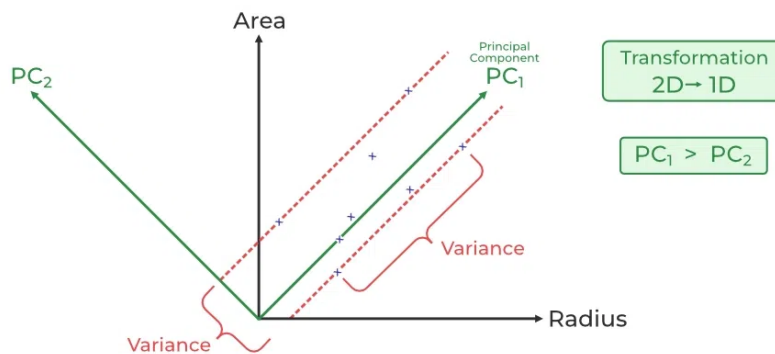


Figure 1: El método de PCA.

4.1 Estandarización

Primero, necesitamos estandarizar nuestro conjunto de datos para asegurarnos de que cada variable tenga una media de 0 y una desviación estándar de 1.

$$Z = \frac{X - \mu}{\sigma}$$

donde:

- X son las características independientes,
- μ es la media de las características independientes,
- σ es la desviación estándar de las características independientes.

4.2 Cálculo de la matriz de covariancia

La covariancia mide la fuerza de la variabilidad conjunta entre dos o más variables, indicando cuánto cambian en relación entre sí. Para encontrar la covariancia, podemos usar la fórmula:

$$\text{cov}(x_1, x_2) = \frac{1}{n-1} \sum_{i=1}^n (x_{1,i} - \bar{x}_1)(x_{2,i} - \bar{x}_2)$$

Los valores de la covariancia pueden ser:

- Positivos: A medida que x_1 aumenta, x_2 también aumenta.
- Negativos: A medida que x_1 aumenta, x_2 disminuye.
- Cero: No hay relación directa.

4.3 Cálculo de valores y vectores propios

A continuación, debemos calcular los valores propios (λ) y los vectores propios de la matriz de covariancia para identificar los componentes principales. Sea A una matriz cuadrada $n \times n$ y X un vector no nulo para el cual:

$$AX = \lambda X$$

donde λ es el valor propio y X es el vector propio correspondiente de la matriz A .

4.4 Selección de los componentes principales

Una vez que tenemos los valores propios, seleccionamos **los k componentes principales** que tienen los valores propios más altos. Estos componentes explican la mayor parte de la varianza en los datos.

4.5 Proyección de los datos

Proyectamos los datos originales en el espacio de los componentes principales para obtener una representación en dimensiones reducidas:

$$Y = XW$$

donde X es la matriz original de datos y W es la matriz de vectores propios seleccionados.

5 Implementación del PCA

A continuación, vamos a implementar PCA paso a paso:

```
from sklearn.datasets import load_breast_cancer
cancer = load_breast_cancer(as_frame=True)
df = cancer.frame
X = df[cancer['feature_names']]
```

Ahora aplicaremos el primer paso, que es estandarizar los datos. Para ello, primero calcularemos la media y la desviación estándar de cada característica en el espacio de características.

```
X_mean = X.mean()
X_std = X.std()
Z = (X - X_mean) / X_std
```

La matriz de covarianza nos ayuda a visualizar la dependencia entre dos características en el espacio de características.

```
c = Z.cov()
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(12, 12))
sns.heatmap(c)
plt.show()
```

Ahora calcularemos los valores propios y los vectores propios para nuestro espacio de características, que son cruciales para identificar los componentes principales de nuestro espacio de características.

```
eigenvalues, eigenvectors = np.linalg.eig(c)
print('Valores propios:\n', eigenvalues)
print('Forma de los valores propios:', eigenvalues.shape)
print('Forma de los vectores propios:', eigenvectors.shape)
```

Ordena los valores propios en orden descendente y organiza los vectores propios correspondientes de manera adecuada.

```
idx = eigenvalues.argsort()[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]
```

La **varianza explicada** es el término que nos da una idea de la cantidad de varianza total que ha sido retenida al seleccionar los componentes principales en lugar del espacio original de características.

```

explained_var = np.cumsum(eigenvalues) / np.sum(eigenvalues)
plt.plot(range(1, len(explained_var) + 1), explained_var)
plt.xlabel("Número de componentes")
plt.ylabel("Varianza explicada acumulada")
plt.grid()
plt.show()

```

Ahora vamos a determinar el número de componentes principales. Aquí consideramos una varianza explicada mayor o igual al 50%.

```
n_components = np.argmax(explained_var >= 0.50) + 1
```

Y finalmente nos falta proyectar los datos en los componentes principales seleccionados. La matriz de proyección es una matriz de vectores propios correspondientes a los valores propios más grandes de la matriz de covarianza de los datos.

```

u = eigenvectors[:, :n_components]
pca_component = pd.DataFrame(u,
                              index = cancer['feature_names'],
                              columns = ['PC1', 'PC2'])
plt.figure(figsize=(5, 7))
sns.heatmap(pca_component, annot=True, cmap='coolwarm')
plt.show()

```

Esta matriz nos dice qué variables originales son más relevantes en la nueva representación de PCA. Así el PC1 y PC2 forman un nuevo espacio de características donde podemos visualizar y clasificar mejor los tumores.

Finalmente, proyectamos nuestro conjunto de datos usando la fórmula:

$$P_i(u) = P_i \cdot u$$

La reducción de dimensionalidad se logra reteniendo solo aquellos ejes que explican la mayor parte de la varianza y descartando el resto.

En Python, necesitamos usar el producto punto, denotado por @, que multiplica la matriz de datos estandarizada Z por la matriz de eigenvectores `pca_component`, lo que proyecta los datos originales en los nuevos componentes principales.

```

Z_pca = Z @ pca_component
Z_pca.columns = ['PCA1', 'PCA2']
print(Z_pca)

```

Los vectores propios de la matriz de covarianza de los datos se denominan **ejes principales** de los datos, y la proyección de las instancias de datos sobre estos

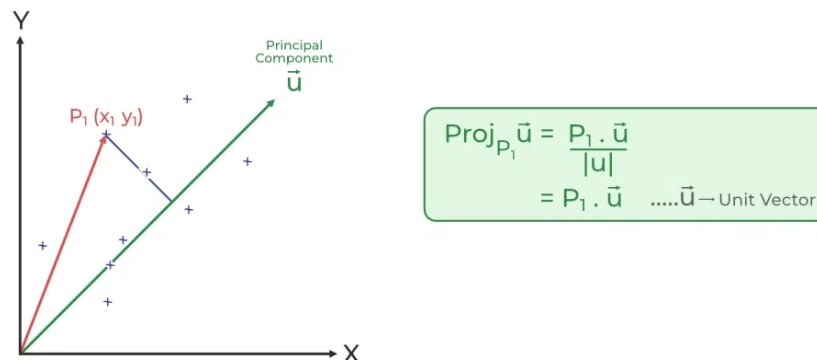


Figure 2: Ejes de PCA.

ejes se denominan los **componentes principales**. La reducción de dimensionalidad se logra reteniendo solo aquellos ejes (dimensiones) que explican la mayor parte de la varianza.

Los PCA1 y PCA2 son ortogonales, y esto significa que:

```
print(Z_pca[['PCA1', 'PCA2']].corr())
```

y los tumores malignos y benignos se separan en el espacio de PCA:

```
Z_pca['target'] = df['target']
plt.figure(figsize=(8,8))
sns.scatterplot(data=Z_pca, x="PCA1", y="PCA2", hue="target")
plt.xlabel('PCA1')
plt.ylabel('PCA2')
plt.show()
```

Y finalmente, necesitamos definir los umbrales basados en la mediana de PCA1 y PCA2 en los tumores malignos, generar las predicciones usando umbrales en PCA1 y PCA2, clasificando como maligno (0) si al menos una de las dos condiciones se cumple, calcular las métricas para PCA1 y PCA2 y comparar con los valores de worst concave points.

```
umbral1 = Z_pca[Z_pca['target'] == 0]['PCA1'].median()
umbral2 = Z_pca[Z_pca['target'] == 0]['PCA2'].median()
Z_pca['pred_pca'] = ((Z_pca['PCA1'] >= umbral1)
| (Z_pca['PCA2'] >= umbral2)).astype(int)
cm_pca12 = confusion_matrix(Z_pca['target'], Z_pca['pred_pca'])
print(accuracy_score(Z_pca['target'], Z_pca['pred_pca']))
```

```
print(precision_score(Z_pca['target'], Z_pca['pred_pca']))
print(recall_score(Z_pca['target'], Z_pca['pred_pca']))
print(f1_score(Z_pca['target'], Z_pca['pred_pca']))
```

Usar dos componentes principales (PCA1 y PCA2) mejora la clasificación notablemente!

6 Implementación del PCA en Scikit-learn

La implementación del PCA en Scikit-learn para solo 2 componentes:

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
pca.fit(Z)
x_pca = pca.transform(Z)
df_pca1 = pd.DataFrame(x_pca,
                        columns=['PC{}'.format(i+1)
                                for i in range(n_components)])
plt.figure(figsize=(8, 6))
plt.scatter(x_pca[:, 0], x_pca[:, 1], c=cancer['target'])
plt.xlabel('PCA1')
plt.ylabel('PCA2')
plt.show()
```

7 Tarea: PCA para Iris

Aplica el método de PCA al conjunto de datos de Iris para demostrar cómo se puede reducir de 4 dimensiones a 2, manteniendo la mayor parte de la información (porcentaje de la varianza total). Para cargar el conjunto en Scikit-learn, usa:

```
from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data
y = iris.target
```